Page Logic Explanation

You can submit the following as your **Logic Explanation**:

State Management — Counter & Theme Switcher

The purpose of this task was to learn how application data can be shared between multiple Flutter screens or tabs without storing separate copies of the same data in each widget. I implemented the task using the **Provider** state-management package and Flutter's `ChangeNotifier` class.

A central `AppState` class stores two pieces of shared application data: an integer counter and a Boolean value representing whether dark mode is enabled. The class extends `ChangeNotifier`, allowing changes to the data to be communicated to widgets that are listening to the provider. Methods such as `incrementCounter()`, `decrementCounter()`, `resetCounter()`, and `toggleTheme()` modify the stored values and then call `notifyListeners()`.

The `ChangeNotifierProvider` is placed above `MaterialApp` in `main.dart`. This makes one instance of `AppState` available throughout the application rather than creating separate state values for individual pages.

The State Management screen contains two tabs: **Counter** and **Theme**. The Counter tab reads the counter using `context.watch<AppState>()`. When the Increase or Decrease buttons are pressed, methods inside `AppState` update the counter. Provider then automatically rebuilds widgets that depend on that value.

The Theme tab reads the same `AppState` object. A `Switch` changes the Boolean `isDarkMode` property, which controls the `ThemeMode` of `MaterialApp`. Because the theme state exists above the individual screens, changing it affects the entire application.

The Counter value is also displayed on the Theme tab. For example, if the counter is increased to 5 and the user moves to the Theme tab, the value remains 5. Returning to the Counter tab still shows 5. Similarly, the selected light or dark theme remains active while switching between tabs.

This demonstrates the main purpose of state management: **maintaining one shared source of data that different parts of an application can read and modify consistently.**